Singleton

This chapter is an anomaly. Every other chapter in this book shows you how to use a design pattern. This chapter shows you how *not* to use one.

Despite noble intentions, the Singleton GoF pattern described by the Gang of Four usually does more harm than good. They stress that the pattern should be used sparingly, but that message was often lost in translation to the game industry.

Like any pattern, using Singleton where it doesn't belong is about as helpful as treating a bullet wound with a splint. Since it's so overused, most of this chapter will be about *avoiding* singletons, but first, let's go over the pattern itself.

> When much of the industry moved to object-oriented programming from C, one problem they ran into was "how do I get an instance?" They had some method they wanted to call but didn't have an instance of the object that provides that method in hand. Singletons (in other words, making it global) were an easy way out.

The Singleton Pattern

Design Patterns summarizes Singleton like this:

> *Ensure a class has one instance, and provide a global point of access to it.*

We'll split that at "and" and consider each half separately.

Restricting a class to one instance

There are times when a class cannot perform correctly if there is more than one instance of it. The common case is when the class interacts with an external system that maintains its own global state.

Consider a class that wraps an underlying file system API. Because file operations can take a while to complete, our class performs operations asynchronously. This means multiple operations can be running concurrently, so they must be coordinated with each other. If we start one call to create a file and another one to delete that same file, our wrapper needs to be aware of both to make sure they don't interfere with each other.

To do this, a call into our wrapper needs to have access to every previous operation. If users could freely create instances of our class, one instance would have no way of knowing about operations that other instances started. Enter the singleton. It provides a way for a class to ensure at compile time that there is only a single instance of the class.

Providing a global point of access

Several different systems in the game will use our file system wrapper: logging, content loading, game state saving, etc. If those systems can't create their own instances of our file system wrapper, how can they get ahold of one?

Singleton provides a solution to this too. In addition to creating the single instance, it also provides a globally available method to get it. This way, anyone anywhere can get their paws on our blessed instance. All together, the classic implementation looks like this:

```
class FileSystem
{
public:
  static FileSystem& instance()
  {
    // Lazy initialize.
    if (instance_ == NULL) instance_ = new FileSystem();
    return *instance_;
  }

private:
  FileSystem() {}

  static FileSystem* instance_;
};
```

The static `instance_` member holds an instance of the class, and the private constructor ensures that it is the *only* one. The public static `instance()` method grants access to the instance from anywhere in the codebase. It is also responsible for instantiating the singleton instance lazily the first time someone asks for it.

A modern take looks like this:

```
class FileSystem
{
public:
  static FileSystem& instance()
  {
    static FileSystem *instance = new FileSystem();
    return *instance;
  }

private:
  FileSystem() {}
};
```

C++11 mandates that the initializer for a local static variable is only run once, even in the presence of concurrency. So, assuming you've got a modern C++ compiler, this code is thread-safe where the first example is not.

> Of course, the thread-safety of your singleton class itself is an entirely different question! This just ensures that its *initialization* is.

Why We Use It

It seems we have a winner. Our file system wrapper is available wherever we need it without the tedium of passing it around everywhere. The class itself cleverly ensures we won't make a mess of things by instantiating a couple of instances. It's got some other nice features too:

- **It doesn't create the instance if no one uses it.** Saving memory and CPU cycles is always good. Since the singleton is initialized only when it's first accessed, it won't be instantiated at all if the game never asks for it.

- **It's initialized at runtime.** A common alternative to Singleton is a class with static member variables. I like simple solutions, so I use static classes instead of singletons when possible, but there's one limitation static members have: automatic initialization. The compiler initializes statics before `main()` is called. This means they can't use information known only once the program is up and running (for example, configuration loaded from a file). It also means they can't reliably depend on each other — the compiler does not guarantee the order in which statics are initialized relative to each other.

  Lazy initialization solves both of those problems. The singleton will be initialized as late as possible, so by that time any information it needs should be available. As long as they don't have circular dependencies, one singleton can even refer to another when initializing itself.

- **You can subclass the singleton.** This is a powerful but often overlooked capability. Let's say we need our file system wrapper to be cross-platform. To make this work, we want it to be an abstract interface for a file system with subclasses that implement the interface for each platform. Here is the base class:

```
class FileSystem
{
public:
  virtual ~FileSystem() {}
  virtual char* readFile(char* path) = 0;
  virtual void  writeFile(char* path, char* contents) = 0;
};
```

  Then we define derived classes for a couple of platforms:

```
class PS3FileSystem : public FileSystem
{
public:
  virtual char* readFile(char* path)
  {
    // Use Sony file IO API...
  }

  virtual void writeFile(char* path, char* contents)
  {
    // Use sony file IO API...
  }
};

class WiiFileSystem : public FileSystem
{
public:
  virtual char* readFile(char* path)
  {
    // Use Nintendo file IO API...
  }

  virtual void writeFile(char* path, char* contents)
  {
    // Use Nintendo file IO API...
  }
};
```

  Next, we turn `FileSystem` into a singleton:

```
class FileSystem
{
public:
```

```
  static FileSystem& instance();

  virtual ~FileSystem() {}
  virtual char* readFile(char* path) = 0;
  virtual void  writeFile(char* path, char* contents) = 0;

protected:
  FileSystem() {}
};
```

The clever part is how the instance is created:

```
FileSystem& FileSystem::instance()
{
  #if PLATFORM == PLAYSTATION3
    static FileSystem *instance = new PS3FileSystem();
  #elif PLATFORM == WII
    static FileSystem *instance = new WiiFileSystem();
  #endif

  return *instance;
}
```

With a simple compiler switch, we bind our file system wrapper to the appropriate concrete type. Our entire codebase can access the file system using `FileSystem::instance()` without being coupled to any platform-specific code. That coupling is instead encapsulated within the implementation file for the `FileSystem` class itself.

This takes us about as far as most of us go when it comes to solving a problem like this. We've got a file system wrapper. It works reliably. It's available globally so every place that needs it can get to it. It's time to check in the code and celebrate with a tasty beverage.

Why We Regret Using It

In the short term, the Singleton pattern is relatively benign. Like many design choices, we pay the cost in the long term. Once we've cast a few unnecessary singletons into cold hard code, here's the trouble we've bought ourselves:

It's a global variable

When games were still written by a couple of guys in a garage, pushing the hardware was more important than ivory-tower software engineering principles. Old-school C and assembly coders used globals and statics without any trouble and shipped good games. As games got bigger and more complex, architecture and maintainability started to become the bottleneck. We struggled to ship games not because of hardware limitations, but because of *productivity* limitations.

So we moved to languages like C++ and started applying some of the hard-earned wisdom of our software engineer forebears. One lesson we learned is that global variables are bad for a variety of reasons:

- **They make it harder to reason about code.** Say we're tracking down a bug in a function someone else wrote. If that function doesn't touch any global state, we can wrap our heads around it just by understanding the body of the function and the arguments being passed to it.

  Computer scientists call functions that don't access or modify global state "pure" functions. Pure functions are easier to reason about, easier for the compiler to

optimize, and let you do neat things like memoization where you cache and reuse the results from previous calls to the function.

While there are challenges to using purity exclusively, the benefits are enticing enough that computer scientists have created languages like Haskell that *only* allow pure functions.

Now, imagine right in the middle of that function is a call to `SomeClass::getSomeGlobalData()`. To figure out what's going on, we have to hunt through the entire codebase to see what touches that global data. You don't really hate global state until you've had to `grep` a million lines of code at three in the morning trying to find the one errant call that's setting a static variable to the wrong value.

- **They encourage coupling.** The new coder on your team isn't familiar with your game's beautifully maintainable loosely coupled architecture, but he's just been given his first task: make boulders play sounds when they crash onto the ground. You and I know we don't want the physics code to be coupled to *audio* of all things, but he's just trying to get his task done. Unfortunately for us, the instance of our `AudioPlayer` is globally visible. So, one little `#include` later, and our new guy has compromised a carefully constructed architecture.

  Without a global instance of the audio player, even if he *did* `#include` the header, he still wouldn't be able to do anything with it. That difficulty sends a clear message to him that those two modules should not know about each other and that he needs to find another way to solve his problem. *By controlling access to instances, you control coupling.*

- **They aren't concurrency-friendly.** The days of games running on a simple single-core CPU are pretty much over. Code today must at the very least *work* in a multi-threaded way even if it doesn't take full advantage of concurrency. When we make something global, we've created a chunk of memory that every thread can see and poke at, whether or not they know what other threads are doing to it. That path leads to deadlocks, race conditions, and other hell-to-fix thread-synchronization bugs.

Issues like these are enough to scare us away from declaring a global variable, and thus the Singleton pattern too, but that still doesn't tell us how we *should* design the game. How do you architect a game without global state?

There are some extensive answers to that question (most of this book in many ways *is* an answer to just that), but they aren't apparent or easy to come by. In the meantime, we have to get games out the door. The Singleton pattern looks like a panacea. It's in a book on object-oriented design patterns, so it *must* be architecturally sound, right? And it lets us design software the way we have been doing for years.

Unfortunately, it's more placebo than cure. If you scan the list of problems that globals cause, you'll notice that the Singleton pattern doesn't solve any of them. That's because a singleton *is* global state—it's just encapsulated in a class.

It solves two problems even when you just have one

The word "and" in the Gang of Four's description of Singleton is a bit strange. Is this pattern a solution to one problem or two? What if we have only one of those? Ensuring a single instance is useful, but who says we want to let *everyone* poke at

it? Likewise, global access is convenient, but that's true even for a class that allows multiple instances.

The latter of those two problems, convenient access, is almost always why we turn to the Singleton pattern. Consider a logging class. Most modules in the game can benefit from being able to log diagnostic information. However, passing an instance of our `Log` class to every single function clutters the method signature and distracts from the intent of the code.

The obvious fix is to make our `Log` class a singleton. Every function can then go straight to the class itself to get an instance. But when we do that, we inadvertently acquire a strange little restriction. All of a sudden, we can no longer create more than one logger.

At first, this isn't a problem. We're writing only a single log file, so we only need one instance anyway. Then, deep in the development cycle, we run into trouble. Everyone on the team has been using the logger for their own diagnostics, and the log file has become a massive dumping ground. Programmers have to wade through pages of text just to find the one entry they care about.

We'd like to fix this by partitioning the logging into multiple files. To do this, we'll have separate loggers for different game domains: online, UI, audio, gameplay. But we can't. Not only does our `Log` class no longer allow us to create multiple instances, that design limitation is entrenched in every single call site that uses it:

```
Log::instance().write("Some event.");
```

In order to make our `Log` class support multiple instantiation (like it originally did), we'll have to fix both the class itself and every line of code that mentions it. Our convenient access isn't so convenient anymore.

> It could be even worse than this. Imagine your `Log` class is in a library being shared across several *games*. Now, to change the design, you'll have to coordinate the change across several groups of people, most of whom have neither the time nor the motivation to fix it.

Lazy initialization takes control away from you

In the desktop PC world of virtual memory and soft performance requirements, lazy initialization is a smart trick. Games are a different animal. Initializing a system can take time: allocating memory, loading resources, etc. If initializing the audio system takes a few hundred milliseconds, we need to control when that's going to happen. If we let it lazy-initialize itself the first time a sound plays, that could be in the middle of an action-packed part of the game, causing visibly dropped frames and stuttering gameplay.

Likewise, games generally need to closely control how memory is laid out in the heap to avoid fragmentation. If our audio system allocates a chunk of heap when it initializes, we want to know *when* that initialization is going to happen, so that we can control *where* in the heap that memory will live.

> See Object Pool → for a detailed explanation of memory fragmentation.

Because of these two problems, most games I've seen don't rely on lazy initialization. Instead, they implement the Singleton pattern like this:

```
class FileSystem
{
public:
  static FileSystem& instance() { return instance_; }

private:
  FileSystem() {}

  static FileSystem instance_;
};
```

That solves the lazy initialization problem, but at the expense of discarding several singleton features that *do* make it better than a raw global variable. With a static instance, we can no longer use polymorphism, and the class must be constructible at static initialization time. Nor can we free the memory that the instance is using when not needed.

Instead of creating a singleton, what we really have here is a simple static class. That isn't necessarily a bad thing, but if a static class is all you need, why not get rid of the `instance()` method entirely and use static functions instead? Calling `Foo::bar()` is simpler than `Foo::instance().bar()`, and also makes it clear that you really are dealing with static memory.

> The usual argument for choosing singletons over static classes is that if you decide to change the static class into a non-static one later, you'll need to fix every call site. In theory, you don't have to do that with singletons because you could be passing the instance around and calling it like a normal instance method.
>
> In practice, I've never seen it work that way. Everyone just does `Foo::instance().bar()` in one line. If we changed Foo to not be a singleton, we'd still have to touch every call site. Given that, I'd rather have a simpler class and a simpler syntax to call into it.

What We Can Do Instead

If I've accomplished my goal so far, you'll think twice before you pull Singleton out of your toolbox the next time you have a problem. But you still have a problem that needs solving. What tool *should* you pull out? Depending on what you're trying to do, I have a few options for you to consider, but first...

See if you need the class at all

Many of the singleton classes I see in games are "managers" — those nebulous classes that exist just to babysit other objects. I've seen codebases where it seems like *every* class has a manager: Monster, MonsterManager, Particle, ParticleManager, Sound, SoundManager, ManagerManager. Sometimes, for variety, they'll throw a "System" or "Engine" in there, but it's still the same idea.

While caretaker classes are sometimes useful, often they just reflect unfamiliarity with OOP. Consider these two contrived classes:

```
class Bullet
{
public:
  int getX() const { return x_; }
  int getY() const { return y_; }

  void setX(int x) { x_ = x; }
  void setY(int y) { y_ = y; }
```

```
private:
  int x_, y_;
};

class BulletManager
{
public:
  Bullet* create(int x, int y)
  {
    Bullet* bullet = new Bullet();
    bullet->setX(x);
    bullet->setY(y);

    return bullet;
  }

  bool isOnScreen(Bullet& bullet)
  {
    return bullet.getX() >= 0 &&
           bullet.getX() < SCREEN_WIDTH &&
           bullet.getY() >= 0 &&
           bullet.getY() < SCREEN_HEIGHT;
  }

  void move(Bullet& bullet)
  {
    bullet.setX(bullet.getX() + 5);
  }
};
```

Maybe this example is a bit dumb, but I've seen plenty of code that reveals a design just like this after you scrape away the crusty details. If you look at this code, it's natural to think that `BulletManager` should be a singleton. After all, anything that has a `Bullet` will need the manager too, and how many instances of `BulletManager` do you need?

The answer here is *zero*, actually. Here's how we solve the "singleton" problem for our manager class:

```
class Bullet
{
public:
  Bullet(int x, int y) : x_(x), y_(y) {}

  bool isOnScreen()
  {
    return x_ >= 0 && x_ < SCREEN_WIDTH &&
           y_ >= 0 && y_ < SCREEN_HEIGHT;
  }

  void move() { x_ += 5; }

private:
  int x_, y_;
};
```

There we go. No manager, no problem. Poorly designed singletons are often "helpers" that add functionality to another class. If you can, just move all of that behavior into the class it helps. After all, OOP is about letting objects take care of themselves.

Outside of managers, though, there are other problems where we'd reach to Singleton for a solution. For each of those problems, there are some alternative solutions to consider.

To limit a class to a single instance

This is one half of what the Singleton pattern gives you. As in our file system example, it can be critical to ensure there's only a single instance of a class. However, that doesn't necessarily mean we also want to provide *public*, *global*

access to that instance. We may want to restrict access to certain areas of the code or even make it private to a single class. In those cases, providing a public global point of access weakens the architecture.

> For example, we may be wrapping our file system wrapper inside *another* layer of abstraction.

We want a way to ensure single instantiation *without* providing global access. There are a couple of ways to accomplish this. Here's one:

```
class FileSystem
{
public:
  FileSystem()
  {
    assert(!instantiated_);
    instantiated_ = true;
  }

  ~FileSystem() { instantiated_ = false; }

private:
  static bool instantiated_;
};

bool FileSystem::instantiated_ = false;
```

This class allows anyone to construct it, but it will assert and fail if you try to construct more than one instance. As long as the right code creates the instance first, then we've ensured no other code can either get at that instance or create their own. The class ensures the single instantiation requirement it cares about, but it doesn't dictate how the class should be used.

> An *assertion* function is a way of embedding a contract into your code. When `assert()` is called, it evaluates the expression passed to it. If it evaluates to `true`, then it does nothing and lets the game continue. If it evaluates to `false`, it immediately halts the game at that point. In a debug build, it will usually bring up the debugger or at least print out the file and line number where the assertion failed.
>
> An `assert()` means, "I assert that this should always be true. If it's not, that's a bug and I want to stop *now* so you can fix it." This lets you define contracts between regions of code. If a function asserts that one of its arguments is not `NULL`, that says, "The contract between me and the caller is that I will not be passed `NULL`."
>
> Assertions help us track down bugs as soon as the game does something unexpected, not later when that error finally manifests as something visibly wrong to the user. They are fences in your codebase, corralling bugs so that they can't escape from the code that created them.

The downside with this implementation is that the check to prevent multiple instantiation is only done at *runtime*. The Singleton pattern, in contrast, guarantees a single instance at compile time by the very nature of the class's structure.

To provide convenient access to an instance

Convenient access is the main reason we reach for singletons. They make it easy to get our hands on an object we need to use in a lot of different places. That ease comes at a cost, though—it becomes equally easy to get our hands on the object in places where we *don't* want it being used.

The general rule is that we want variables to be as narrowly scoped as possible while still getting the job done. The smaller the scope an object has, the fewer places we need to keep in our head while we're working with it. Before we take the shotgun approach of a singleton object with *global* scope, let's consider other ways our codebase can get access to an object:

- **Pass it in.** The simplest solution, and often the best, is to simply pass the object you need as an argument to the functions that need it. It's worth considering before we discard it as too cumbersome.

  > Some use the term "dependency injection" to refer to this. Instead of code reaching *out* and finding its dependencies by calling into something global, the dependencies are pushed *in* to the code that needs it through parameters. Others reserve "dependency injection" for more complex ways of providing dependencies to code.

  Consider a function for rendering objects. In order to render, it needs access to an object that represents the graphics device and maintains the render state. It's very common to simply pass that in to all of the rendering functions, usually as a parameter named something like `context`.

  On the other hand, some objects don't belong in the signature of a method. For example, a function that handles AI may need to also write to a log file, but logging isn't its core concern. It would be strange to see `Log` show up in its argument list, so for cases like that we'll want to consider other options.

  > The term for things like logging that appear scattered throughout a codebase is "cross-cutting concern". Handling cross-cutting concerns gracefully is a continuing architectural challenge, especially in statically typed languages.
  >
  > Aspect-oriented programming was designed to address these concerns.

- **Get it from the base class.** Many game architectures have shallow but wide inheritance hierarchies, often only one level deep. For example, you may have a base `GameObject` class with derived classes for each enemy or object in the game. With architectures like this, a large portion of the game code will live in these "leaf" derived classes. This means that all these classes already have access to the same thing: their `GameObject` base class. We can use that to our advantage:

  ```
  class GameObject
  {
  protected:
    Log& getLog() { return log_; }

  private:
    static Log& log_;
  };

  class Enemy : public GameObject
  {
    void doSomething()
    {
      getLog().write("I can log!");
    }
  };
  ```

  This ensures nothing outside of `GameObject` has access to its `Log` object, but every derived entity does using `getLog()`. This pattern of letting derived objects implement themselves in terms of protected methods provided to them is covered in the Subclass Sandbox→ chapter.

This raises the question, "how does `GameObject` get the `Log` instance?" A simple solution is to have the base class simply create and own a static instance.

If you don't want the base class to take such an active role, you can provide an initialization function to pass it in or use the Service Locator→ pattern to find it.

- **Get it from something already global.** The goal of removing *all* global state is admirable, but rarely practical. Most codebases will still have a couple of globally available objects, such as a single `Game` or `World` object representing the entire game state.

  We can reduce the number of global classes by piggybacking on existing ones like that. Instead of making singletons out of `Log`, `FileSystem`, and `AudioPlayer`, do this:

```
class Game
{
public:
  static Game& instance() { return instance_; }

  // Functions to set log_, et. al. ...

  Log&         getLog()         { return *log_; }
  FileSystem&  getFileSystem()  { return *fileSystem_; }
  AudioPlayer& getAudioPlayer() { return *audioPlayer_; }

private:
  static Game instance_;

  Log         *log_;
  FileSystem  *fileSystem_;
  AudioPlayer *audioPlayer_;
};
```

  With this, only `Game` is globally available. Functions can get to the other systems through it:

```
Game::instance().getAudioPlayer().play(VERY_LOUD_BANG);
```

  Purists will claim this violates the Law of Demeter. I claim that's still better than a giant pile of singletons.

  If, later, the architecture is changed to support multiple `Game` instances (perhaps for streaming or testing purposes), `Log`, `FileSystem`, and `AudioPlayer` are all unaffected — they won't even know the difference. The downside with this, of course, is that more code ends up coupled to `Game` itself. If a class just needs to play sound, our example still requires it to know about the world in order to get to the audio player.

  We solve this with a hybrid solution. Code that already knows about `Game` can simply access `AudioPlayer` directly from it. For code that doesn't, we provide access to `AudioPlayer` using one of the other options described here.

- **Get it from a Service Locator.** So far, we're assuming the global class is some regular concrete class like `Game`. Another option is to define a class whose sole reason for being is to give global access to objects. This common pattern is called a Service Locator→ and gets its own chapter.

What's Left for Singleton

The question remains, where *should* we use the real Singleton pattern? Honestly, I've never used the full Gang of Four implementation in a game. To ensure single instantiation, I usually simply use a static class. If that doesn't work, I'll use a static flag to check at runtime that only one instance of the class is constructed.

There are a couple of other chapters in this book that can also help here. The Subclass Sandbox pattern gives instances of a class access to some shared state without making it globally available. The Service Locator pattern *does* make an object globally available, but it gives you more flexibility with how that object is configured.

© 2009-2014 Robert Nystrom